

Why FHE Still Needs an Authenticated Encrypted Channel

And whether an integrity-only scheme would be enough

Project	Privacy-Preserving Collaborative IDS for IoV (LGSi)
Question from	Design review, Month 5
Date	2026-08-09
Verdict	Both layers are required. The integrity-only proposal is half right, and it exposes a real gap in the current design.

“Anyhow the encrypted data is only going to the server side, so why do we need an AES channel? And if integrity is the problem, can we not just use one integrity / authenticity verification scheme?”

1. Short answer

Both layers are needed, but **not for the reason usually given**. The second half of the question is the sharper one, and it is largely correct: the missing property really is authentication and integrity, and a signature scheme supplies most of it. Plain confidentiality-on-the-wire is *not* what justifies the channel — FHE already hides the payload from anyone listening.

Two things still force an encrypted channel rather than a signature alone:

- **The masked key shares must not be public.** FheFL has each vehicle upload ss_u alongside its ciphertext. Anyone who collects all of them can reconstruct the aggregate decryption key. Broadcasting those in the clear hands every passive listener the server's capability.
- **A bare signature is a tracking beacon.** A signature identifies its signer to anyone with a radio. In a vehicular network that converts the protocol into a location-tracking oracle — which attacks the very privacy the project exists to protect.

And the question surfaces a genuine defect. AES-GCM authenticates with a *symmetric* tag, so the server — which holds the same key — could forge a vehicle's update. The system therefore cannot prove which vehicle submitted a poisoned model. That is a real gap, and the fix is exactly the mechanism proposed in the question: add a per-message signature. See section 7.

2. “Security” is not one property

Most confusion here comes from treating security as a single thing that either is or is not present. It decomposes into at least five independent properties, and a given primitive supplies some and not others.

Property	Plain meaning	Who is the attacker?
Confidentiality	Nobody can read the content	Eavesdropper, or the recipient itself
Authentication	The message really came from who it claims	Impersonator
Integrity	The message was not modified in flight	On-path attacker
Freshness	This is not an old message resent	Replay attacker
Non-repudiation	The sender cannot later deny sending it, and a third party can verify who did	A lying sender, or a lying server

The decisive asymmetry. Transport encryption protects data *in transit*. It can never protect data *from the recipient*, because the recipient holds the key. The server is the one party AES structurally cannot defend you against — and the server is precisely the adversary this project cares about. That is the entire reason FHE is in the design at all.

3. What each layer actually provides

Threat	Adversary	AES-GCM channel	FHE
Eavesdropping on the radio	network	yes	yes
Server reads your individual update	server	NO - it holds the key	YES
Impersonating a vehicle	network	yes	no
Modifying a ciphertext in flight	network	yes	NO - malleable by design
Replaying an old update	network	yes, with round binding	no

Threat	Adversary	AES-GCM channel	FHE
Registering 500 fake vehicles (Sybil)	attacker	yes, via identity keys	no
Proving who sent a poisoned update	server or vehicle	NO - symmetric tag	no

The two **NOs** in the FHE column are the ones that matter, and the reason is section 4.

4. Why FHE cannot supply integrity - malleability

Malleability means an attacker who cannot read a ciphertext can still change it into a ciphertext of a *related, predictable* plaintext. For ordinary encryption this is a serious flaw, and modern schemes are built to prevent it.

Homomorphic encryption **gives malleability up deliberately**. That is the whole point: the server must be able to take $\text{Enc}(a)$ and $\text{Enc}(b)$ and produce $\text{Enc}(a+b)$ without the key. But the same property is available to anyone on the path:

```
attacker intercepts Enc(f)
attacker computes Enc(f) + Enc(bias) = Enc(f + bias)
server receives and aggregates a poisoned update it cannot detect
```

The server cannot notice, because the modified ciphertext is a perfectly valid ciphertext. No property of CKKS prevents this — CKKS is at best IND-CPA, and is explicitly not IND-CCA. **Integrity has to come from somewhere else.** The GCM authentication tag supplies it: change one bit of the frame and verification fails.

This is the cleanest refutation of “FHE is enough”. It is not that FHE happens to lack integrity; it is that FHE **must** lack it in order to be useful. The property that makes homomorphic aggregation possible is the same property an attacker uses to tamper with updates.

5. The proposal: integrity/authenticity only

The proposal is to drop the encrypted channel and instead attach an authenticity tag to each FHE ciphertext — for example an Ed25519 signature over $(\text{round_id} \parallel \text{ciphertext})$. Since FHE already hides the payload, why encrypt it a second time?

5.1 What the proposal gets right

- The FHE ciphertext genuinely is confidential against a passive listener. Wrapping it in AES adds **no** extra confidentiality for that payload.
- A signature over the ciphertext plus the round number delivers authentication, integrity and freshness — the three properties FHE lacks.
- A signature is in one way **stronger** than AES-GCM's tag: it is asymmetric, so it also gives non-repudiation, which a shared-key tag cannot.

So the reasoning is sound. It fails on two things that are specific to this system.

5.2 Objection 1 - the masked key shares must stay secret

In FheFL each vehicle uploads two things per round: the ciphertext $[c^u]$, and its **masked key share** $ss_u = s_u + \text{Sum}_{i=1, \dots, i} s_{ii}$. The server sums those to recover the key that decrypts the aggregate:

$$\text{Sum}_u ss_u = \text{Sum}_u s_u = s$$

The masking makes an individual ss_u useless on its own — but the **set** of them is exactly the server's decryption capability. If they travel in the clear, any passive listener that hears the whole round reconstructs s and becomes a second aggregating party.

Why that breaks the security argument. FheFL's Theorem 1 states that compromising one honest vehicle requires colluding with $U-1$ vehicles. That count assumes the server is the *only* party holding s . Publishing the shares silently adds every eavesdropper to the set of parties holding s — so an attacker who listens to the radio and additionally corrupts $U-2$ vehicles breaks the remaining one, without ever touching the server. The whole collusion analysis has to be redone, and it comes out weaker.

A signature does not fix this, because a signature authenticates a message without hiding it. These values need **confidentiality**, and confidentiality against the network is exactly what an encrypted channel provides.

5.3 Objection 2 - a bare signature is a tracking beacon

A signature identifies its signer. That is its purpose. But it means anyone with a radio receiver can read, from an unencrypted frame:

- **which** vehicle transmitted (from the key or certificate),
- **when** it transmitted,
- **roughly where** it was, from signal strength and direction.

Log that over a city and you have a movement history per vehicle — built out of the privacy-preserving protocol itself. This attacks asset A4 (fleet membership and identity) in the threat model and makes threat P7 (traffic analysis) far worse than it already is.

With an ECDH-derived AES session the identity sits *inside* the encrypted envelope; a listener sees only opaque bytes of roughly constant size. This is not a hypothetical concern in vehicular networking — it is why IEEE 1609.2 defines rotating pseudonym certificates. Location privacy is a first-class requirement in V2X, and a plaintext signature is directly at odds with it.

5.4 Objection 3 - defence in depth

CKKS is young and its security assumptions have moved recently: Li and Micciancio (Eurocrypt 2021) showed CKKS is not IND-CPA^D, meaning released decryption results leak key information. Parameters get revised; attacks improve. If the FHE layer is the *only* thing protecting the payload on the wire, any weakening of it is total. With a transport layer underneath, an FHE weakness still leaves an attacker facing AES-256.

5.5 And it would not be accepted anyway

Automotive V2X deployment is governed by IEEE 1609.2 and the ETSI ITS security standards, which mandate authenticated and, for many message classes, encrypted channels. “We use homomorphic encryption instead” is not an available answer at certification. The channel is going to be there regardless of what the research prototype does.

6. Then why AES-GCM specifically?

Given that both confidentiality and integrity are needed on the wire, the question becomes whether to use two primitives (encrypt, then sign) or one that does both. **AEAD** — Authenticated Encryption with Associated Data — is the standard answer, and AES-256-GCM is the standard AEAD.

Option	Confidentiality	Integrity	Cost	Verdict
Signature only	no	yes	~50 us/msg	Fails objections 1 and 2
Encrypt-then-MAC	yes	yes	two primitives, easy to get wrong	Works, but AEAD exists precisely to remove the footguns
AES-256-GCM (AEAD)	yes	yes	hardware-accelerated, GB/s	Chosen - one primitive, both properties

There is also no efficiency argument for dropping it. AES-GCM runs at gigabytes per second with AES-NI hardware support. Against a measured 18.0 ms for a single encrypted squared-distance operation, and roughly 3.6 MB of ciphertext per vehicle per round, the channel is free in comparison. Removing it saves nothing measurable and costs four security properties.

The associated-data field is the point of GCM here. AEAD lets you bind extra context to the frame without encrypting it. Putting the round number in that field is what makes replay detection work: a captured round-3 frame replayed in round 9 fails verification because the bound round number no longer matches. This is currently **not wired up** — see section 8.

7. The real gap this question exposes

The proposal is right that a signature offers something AES-GCM does not, and the current design does not have it.

GCM's authentication tag is a **symmetric** MAC: vehicle and server share one key. So the server can compute any tag the vehicle could. It follows that:

- the server cannot **prove to a third party** that a given update came from a particular vehicle — it could have produced that frame itself;
- a vehicle caught poisoning the model can plausibly deny it;
- the Byzantine-robustness machinery can down-weight a bad update, but the system has no basis to **attribute** it, and therefore no basis to evict or revoke.

In the current implementation Ed25519 is used only during the handshake, to authenticate the key exchange. It is not applied per message. So the moment the session is established, everything after it rests on a shared symmetric key.

Recommendation, arising directly from this question. Add a per-message Ed25519 signature over (round_id || client_id || ciphertext_digest), carried *inside* the AES-GCM envelope. Inside, not outside — that keeps objection 2 closed, because the identity stays encrypted while still being verifiable by the server. This is roughly a day of work and it converts Byzantine detection into Byzantine **accountability**: the server can then prove which vehicle sent which update, which is what revocation and eviction require.

8. What should change in the code

#	Change	Effort	Closes
1	Bind the round number into the AEAD associated data: <code>channel.encrypt(payload, aad=round_id.to_bytes(8,'big'))</code>	1 hour	Replay (threat S5). Already tracked as P0.2
2	Carry <code>ss_u</code> inside the encrypted envelope, never alongside it	already correct - keep it that way	Objection 1
3	Add a per-message Ed25519 signature inside the envelope	1 day	Non-repudiation; enables eviction and revocation
4	Bind the vehicle's identity key to hardware attestation at enrolment	depends on the embedded track	Sybil (threat S4)

9. Summary

Claim	Verdict
"FHE already encrypts it, so the channel is redundant"	False. FHE protects the update from the SERVER; the channel protects it from the NETWORK. Different adversaries. AES can never protect you from the recipient.
"Integrity is the missing property"	Correct, and it is the core of the issue. FHE is malleable by construction, so it cannot supply integrity - and that malleability is exactly what makes homomorphic aggregation work.
"So a signature scheme alone would do"	No. The masked key shares need confidentiality, not just authenticity, and a plaintext signature turns the protocol into a vehicle-tracking oracle.
"Then the channel should be AEAD"	Yes. AES-256-GCM gives confidentiality and integrity in one primitive, is effectively free next to CKKS, and its associated-data field is what makes replay protection possible.
"But a signature adds something AES-GCM cannot"	Correct, and this is the useful part. GCM's tag is symmetric, so the server could forge a vehicle's update. Per-message signatures are needed for accountability, and are missing today.

Net position. Neither layer is redundant, and the two are not alternatives: FHE answers "can the server read my data", the channel answers "is this really my data, unmodified, and fresh". Layering them is ordinary practice - HTTPS does not make end-to-end encryption pointless in a messaging app, for exactly this reason. Removing FHE would reduce the project to the FedIoV baseline; removing the channel would not simplify it, only break it.

Related documents

- `08_threat_model.md` — threats N1-N3 (network), S4 (Sybil), S5 (replay), P7 (traffic analysis)
- `09_security_specification.md` §6 — the channel construction; §5 — FheFL key sharing
- `04_decision_audit.md` §12 — why removing the channel in Month 4 was a regression
- `12_next_steps.md` — P0.2 (round binding), P0.3 (noise flooding)